

MidnightOS Status

“NEARFall” Feasibility Study — Project Increment 1

IOG/ARC

1 April 2026

Agenda

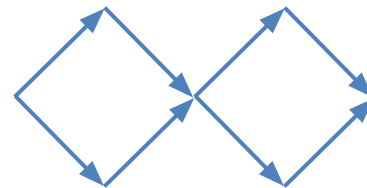
1. Scope, goals, and timeline
2. Summarize the last four weeks of investigations
 - Experiments
 - Technical assessments
 - Findings
3. Collect candid feedback on progress so far
4. Plan
 - Tighter collaboration
 - Course corrections / pivot
 - Next increment

Scope and goals

Goal	Description
Key management	NEAR's key system as inspiration for new Midnight capabilities
Chain abstraction	Derived keys for Midnight, Cardano, Ethereum, Solana, etc.
TPS gains	Improve Midnight throughput, perhaps using NEAR
TEE privacy	TEE as a fast path to adding privacy
Midnight OS platform	NEAR as the substrate for Starstream, Nightstream, Paima, etc.
Feature identification	Identify NEAR capabilities that could benefit Midnight

Option	Approach	Cost
1 — Port	Full re-platforming onto the NEAR stack	Very high
2 — Incorporate software	Extract specific NEAR components	Medium-high
3 — Incorporate ideas	Rebuild NEAR-inspired patterns natively	Medium

Tentative process



Diamond	Phase	Days	Dates	Focus
1 — Foundation	Divergent ← <i>today</i>	1–25	Mar 12 – Apr 5	<i>Broad exploration</i> : generate hypotheses
	Convergent	26–50	Apr 6 – Apr 30	<i>Hypothesis testing</i> : converge on recommendations
2 — Refinement	Divergent	51–75	May 1 – May 25	<i>Refined exploration</i> : regenerate hypotheses
	Convergent	76–100	May 26 – Jun 19	<i>Hypothesis testing</i> : converge on recommendations

<https://github.com/input-output-hk/arc-nearfall-evaluation> 

- Daily journal
- Experiments
- Assessments
- Findings
- Artifacts
- Literature / links

Caveats

- So far, the analyses and experiments are mostly based on public information.
- PDRs, ADRs, benchmark/simulation results, dead ends, etc. might provide information that would change some conclusions.
- Some of the hypotheses outlined here might have already been tested and rejected.

Summary of experiments

Experiment	Platform	Key Finding
test-contract-1	Midnight preprod	Basic wallet operations (mnemonic → sync → transfer) work end-to-end; documentation was incomplete and partly obsolete
midnight-env	Midnight preprod	Node deployment required piecing together multiple outdated sources; official docs are incorrect in several aspects; block fetching was not reliable
node-lan	Midnight (local 7-node cluster)	Consensus was fragile; BEEFY keys apparently cannot be pre-loaded; difficult to work with documentation
shielding-contracts	Midnight preprod	Shielded mint and transfer work, but the SDK has multiple non-obvious shortcomings requiring workarounds (notably a WASM panic in <code>ZswapChainState.tryApply</code>); experienced Lace bug
compact-named-accounts	Midnight preprod	Stealth meta-address registry works end-to-end; however, publishing the ephemeral key on-chain leaks existence of payment events — neither path provides full graph privacy
near-exploration	NEAR testnet	NEAR developer tooling is significantly smoother than Midnight's; account creation, contract deployment, and RPC node setup all work with minimal friction
starstream-compile	Starstream (local)	Starstream compiler produces valid WASM from <code>.star</code> source; the language is functional but early-stage
paima-game-templates	Paima engine	Paima is a real, usable multi-chain rollup engine; templates are game/app oriented, not ZK-proof-based settlement; template was outdated

Index of assessments (1 of 2)

Assessment	Topic	Key Finding
midnight-pallets	Architecture	Midnight combines standard Substrate pallets with custom ones (Kachina, Dust, Minotaur) for ZK verification, fee batteries, and Cardano anchoring
modularity-comparison	Architecture	Substrate is more modular than NEAR on virtually every axis; Option 1 (full port) carries highest risk due to unvalidated 300 TGas ZK-verification limit
contract-to-contract-calls	Architecture	Midnight will eventually support synchronous intra-transaction contract calls; NEAR uses asynchronous inter-block receipts — a fundamental architectural difference
midnight-vs-near-tps	Throughput	Midnight's binding constraint is block size, not proof verification; 500+ TPS may be achievable within Substrate via block-size increases and proof folding
substrate-throughput-techniques	Throughput	No existing Substrate chain demonstrates >50 TPS for ZK-proof-bearing transactions
throughput-constraint-comparison	Throughput	NEAR and Midnight have inverse architectures: NEAR is overhead-heavy with per-shard parallelism; Midnight is payload-efficient with less horizontal scaling
tee-cheat-codes	Privacy	Options available for Midnight, but crypto incompatibilites need resolving

Index of assessments (2 of 2)

Assessment	Topic	Key Finding
midnight-to-near-mapping	State mapping	Per-UTXO/note accounts (Options A/B) rank highest for throughput; coarse-grained options shift the 500+ TPS problem entirely to a bespoke off-chain sequencer
near-key-management	Cryptography	NEAR uses human-readable hierarchical accounts with multiple fine-grained access keys per account, stored directly in the state trie
near-scheduling	Validator ops	NEAR's multi-role validator model distributes load finely but requires significantly higher hardware (48 GB RAM, 2–3 TB storage) than Midnight validators
starstream-nightstream	Execution/proving	Starstream provides UTXO-native WASM execution via coroutines; Nightstream is post-quantum lattice IVC; together they could form a “MidnightOS”, but the Nightstream outer SNARK appears to be still WIP
paima-on-midnight	Integration	Paima could run on Midnight's public layer with moderate effort; privacy integration via ZK attestations is hard due to tension between deterministic re-execution and private state
batching-vs-folding	Proving	AlephBFT replaces only GRANDPA; AURA block production is unchanged, giving ~12–18× finality latency improvement with no raw TPS gain

Emphases during this iteration

Area	Status	Activities
Key management	Explored	Capability survey and initial Midnight applicability analysis
Modularity	Explored	Substrate vs NEAR comparison; findings preliminary
Throughput	Explored	TPS constraints, Substrate techniques, NEAR vs. Midnight comparison
Feature identification	Explored	Catalog of NEAR capabilities potentially applicable to Midnight
TEE integration	Cursory study	Referenced in feature catalog but no independent analysis undertaken
Chain signatures	Cursory study	Referenced in feature catalog; no independent analysis undertaken
Starstream / Nightstream	Experiment only	Compiler experiment run; architecture documented from public sources; no deep analysis
Paima	Experiment only	Template exploration run; architecture documented from public sources; no deep analysis

KEY MANAGEMENT

NEAR key management capabilities

Capability	Definition
Named accounts	Human-readable string account IDs (e.g. <code>alice.near</code>) with a hierarchical namespace; a parent account controls creation of sub-accounts (e.g. <code>app.alice.near</code>)
FullAccess keys	Root keypair with authority over all account operations including key rotation, contract deployment, and fund transfer; loss is catastrophic
FunctionCall / session keys	Scoped keypair restricted to a specific contract and method set with an optional NEAR spending allowance; isolates the root key from day-to-day dApp interaction
DelegateAction / meta-tx	User signs an inner transaction payload; a relay wraps it in an outer transaction and pays the gas; enables gasless onboarding without trusting the relay with the user's key
HD key derivation	Hierarchical deterministic derivation of an account keypair from a BIP-39 master seed; in NEAR all paths are hardened (Ed25519 limitation), so only the wallet layer benefits — no on-chain derivation
Implicit accounts	Account ID is the hex-encoding of an Ed25519 or secp256k1 public key; the account can receive funds before it is explicitly initialised on-chain, simplifying exchange deposits
Promise-based key mgmt	Contracts can add or delete keys on their <i>own</i> account as a promise action; enables social recovery contracts, DAO-controlled custody, and factory-deployed sub-accounts
Chain signatures / MPC	Threshold MPC network (FROST / <code>cait-sith</code>) that holds a distributed key and signs transactions on <i>other</i> chains (Ethereum, Bitcoin, etc.) on behalf of a NEAR account; cross-chain execution without bridging assets

NEAR key management usage

Capability	User Value	Corporate Value	App Areas	Usage Frequency	On-Chain Complexity	Off-Chain Complexity
Named accounts	★★★★★	★★★★★	All	Universal	Low	Low
FullAccess keys	★★★	★★★	All	Universal	Low	Low
FunctionCall keys	★★★★★	★★★★★	dApps, DeFi, enterprise	High	Low	Low
DelegateAction / meta-tx	★★★★★	★★★★★	Onboarding, all	Growing	Medium	Medium
HD key derivation	★★★★★	★★★★	All wallets	Universal	None	Low
Ed25519 implicit accounts	★★★	★★	Onboarding, exchanges	Moderate	Low	Low
Promise-based key mgmt	★★	★★★★	Recovery, DAO, factory	Moderate	Medium	Medium
Multi-sig contract	★★	★★★★★	Treasury, DAO, custody	Moderate	Medium	Medium
Chain Signatures / MPC	★★★★★	★★★★★	Cross-chain DeFi, custody, bridges	Growing	High	High
FastAuth / email recovery	★★★★★	★★	Onboarding	Moderate	None	High

Applicability to Midnight

Capability	Midnight has it?	Off-chain cost	Contract cost	Ledger cost	Philosophy fit	Privacy impact	Security impact
Named accounts	No	Low	Moderate	High	Poor	High degradation	Neutral
FullAccess keys	Yes (equivalent)	—	—	—	High	Avoid public key events	Neutral
FunctionCall / session keys	No	Moderate	Moderate-high	Very high	High (redesign for ZK)	Enhancement if ZK-private	High improvement
DelegateAction / meta-tx	No	Moderate	Moderate	High	High	Low risk (public layer)	Low risk
HD key derivation	Wallet-layer only	Very low	N/A	N/A	High	Enhancement (diversified addrs)	High improvement
Implicit accounts	Yes	Low (voucher alt.)	Moderate	High	Poor	High degradation	Introduces footgun
Promise-based key mgmt	No	N/A	Moderate	Very high	High (redesign for ZK)	Enhancement if ZK-private	High improvement
Chain signatures / MPC	No (bilateral Cardano only)	High (MPC infra)	Moderate (interface)	Very high	Very high	Enhancement (private requests)	BN254/BLS12-381 to clarify

Note that the off-chain vs contract vs ledged costs are mutually exclusive: i.e., any one of them would achieve the capability.

Tentative findings / hypotheses

1. Named accounts are feasible via a registry-like contract, coupled with off-chain infrastructure.
2. W3C DIDs may be preferable to named accounts.
3. Contract patterns / circuit libraries can support function-call keys, multisig, and promised-based key management.
4. Metatransactions are consistent with Midnight intents, where work is already underway by Alan and Nicolas.
5. Midnight already supports full-access keys, HD key derivation, and implicit accounts.

MODULARITY

Design philosophies

Attribute	Substrate / Midnight	NEAR Protocol
Stated goal	Modular blockchain framework	High-performance sharded L1
Primary modularity axis	Vertical component decomposition (pallets)	Horizontal state decomposition (sharding)
Codebase structure	Framework (substrate) + application (midnight-node)	Monolithic (nearcore) + contract layer
Customization model	<i>First-class</i> : swap pallets, runtimes, consensus	<i>Second-class</i> : extend via smart contracts mostly
Core design tension	Modularity vs. performance	Performance vs. customizability

Layer decomposition

Substrate / Midnight

Application Layer

Hoarfrost · ZSwap · Impact ·
Compact · ZKIR

FRAME Pallet Layer

Consensus · Session · Staking ·
Sudo

Substrate Runtime

WASM Runtime · SCALE Encoding ·
Host API

Substrate Client

Networking · RPC · Block Import

- Pallets separate functionality
- Forkless upgrades

NEAR Protocol

Chain Abstraction Layer

NEAR Intents · Chain Signatures ·
OmniBridge

Runtime Layer

WASM Contract Execution ·
Gas Metering · Account Mgmt ·
Receipt Processing

Blockchain Layer

Nightshade Consensus · Sharding ·
Networking · Block/Chunk
Production · State Sync

- Clean blockchain/runtime split
- Intents/chain abstraction is contract-native

Concern about NEAR archival storage

Storage category	Mainnet archival	Testnet archival	Notes
Total	~118 TB	~16 TB	Split-storage architecture (nearcore 1.35.0+)
Cold storage	~115 TB	~15 TB	Spinning disk / cheap persistent volumes
Hot storage	~3 TB SSD	~1.5 TB SSD	Recent epochs; fast random access
Transaction/receipt data	~5–10 TB	—	~1–2 KB/tx × 5.25 B transactions
State snapshot overhead	~108–113 TB	—	<i>Dominant cost — not transaction history</i>

- NEAR requires expensive archival nodes that currently lack a practical cost-recovery model.
- Snapshots of ledger state dominate storage (and perhaps network) resources.
- Validators prune blocks more than 5 epochs (60 hours) old

Consensus pluggability

	Substrate / Midnight	NEAR
Mechanism	Independent pallets (AURA, BABE, GRANDPA, BEEFY)	Integrated Nightshade/Doomslug — not swappable
Swap cost	<i>Low</i> : reconfigure FRAME, recompile	<i>Very high</i> : fork <code>nearcore</code>
Midnight example	Replacing GRANDPA with AlephBFT	N/A
Assessment	✓ Genuinely modular	✗ Monolithic

Runtime / execution pluggability

	Substrate / Midnight	NEAR
Mechanism	WASM runtime, forkless upgrades via on-chain governance	WASM contract execution within fixed runtime
Upgrade path	Runtime logic upgradeable without hard fork	Protocol upgrades require node software release
Custom VMs	Possible: new pallet can introduce new host functions	Not supported at protocol level
Midnight example	Impact interpreter lives in WASM runtime, upgradeable	An Impact-equivalent would be a gas-constrained WASM contract
Assessment	✅ Strong forkless upgrade path	⚠️ Contract-layer extensible; protocol-layer rigid

State storage pluggability

	Substrate / Midnight	NEAR
Storage model	SCALE-encoded Merkle-Patricia Trie; Hoarfrost wraps it	Merkle Patricia Trie, partitioned by shard
State isolation	Single global trie (no native sharding)	Per-shard tries composing a global trie
Custom state structures	Possible via pallet storage items and host API	Fixed per-account key-value store
ZK-specific state	Hoarfrost adds commitment trees, nullifier sets, $O(1)$ cloning	Not natively supported; would require contract-level state
Assessment	✅ Extensible via host API versioning	⚠️ Rigid per-account model; ZK structures non-trivial to add

Scaling model

	Substrate / Midnight	NEAR
Native approach	Single-shard L1; scaling via co-chains and L2 (Midnight City via Paima)	Native sharding: Nightshade splits state/execution across shards
Shard count	N/A at L1; co-chains are separate Substrate nodes	Currently 9+ shards, planned to scale dynamically
Cross-shard latency	N/A at L1; co-chain settlement adds multi-block latency	~1 second per shard hop via receipt routing
State partitioning	Manual (each co-chain has its own state)	Automatic (accounts assigned to shards by ID)
Assessment	⚠ Scaling requires explicit co-chain architecture	✅ Native; sharding is a first-class protocol feature

Tentative findings / hypotheses

1. NEAR's architecture and codebase do not provide sufficient enhancements to justify migrating Midnight to it.
2. Subjectively, NEAR's architecture and design seem less demonstrably robust compared to Substrate.
3. NEAR's transactions are not atomic: they may partially fail and/or partially roll back.
4. NEAR has an excessive storage burden for frequent snapshots of ledger state.

Trusted Execution Environment (TEE)

NEAR's TEE pattern: chain signatures

- *Chain signatures (v1.signer, Aug 2024)*: NEAR accounts control addresses on any external chain via an 8-node threshold MPC network.
- *TEE role*: each MPC node runs inside an enclave and provides a remote attestation.
 - Proves the node is running unmodified signing code.
 - Prevents nodes exfiltrating key shares or censoring requests.
- *MPC role (FROST / cait-sith)*: distributes key shares so no single node can sign unilaterally.
- Neither TEE or MPC alone is sufficient.

NEAR's TEE implications for Midnight

- *Option 2 — Adopt software:* adopt NEAR's cait-sith + TEE stack
 - NEAR's Chain Signatures uses FROST threshold signing (cait-sith library) with TEE attestation
- *Option 3 — Adopt ideas:* build Midnight-native TEE + MPC architecture
 - Design a Midnight-native TEE attestation architecture inspired by NEAR's pattern
 - *Near-term target:* TEE-attested remote proof server
 - *Medium-term:* TEE + FROST threshold signing for a Midnight-native cross-chain signing service
 - *Longer-term:* TEE-attested solvers (prevents front-running) and sequencers (prevents censorship)



Both options require resolving the cryptographic incompatibilities.

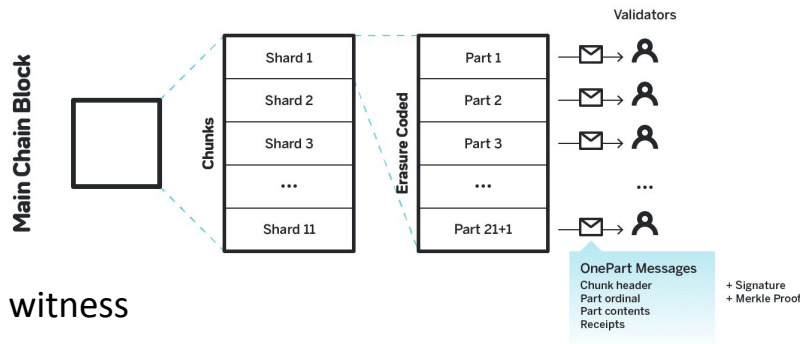
Findings / hypotheses

1. Cryptographic incompatibilities prevent wholesale adoption of NEAR chain signatures.
2. A TEE-attested remote proof server is the feasible for Midnight.
3. TEE attestation can stay off-chain.
4. “TEE cheat codes add privacy quickly.”
 - a. Ship a privacy-claiming feature without a ZK circuit.
 - b. Explicitly a shortcut, not a replacement for ZK.

THROUGHPUT

NEAR's high-throughput architecture

- *Core idea*: horizontal scaling — split global state and execution across parallel shards
- Nightshade Sharding
 - Global state divided into N shards
 - Each shard owns a subset of accounts and contracts
 - Each block contains one “chunk” per shard, produced by that shard's assigned validators
 - Validators track only their assigned shard, not the full global state
- Protocol supports dynamic resharding
- Receipt-Based Async Execution
 - Receipts can spawn further receipts
 - No synchronous cross-shard calls
 - Transactions are not atomic, but receipts are
- Stateless Validation (Nightshade 2.0)
 - Chunk producer executes state and emits a state witness
 - Merkle proofs for every trie read/write
 - Chunk validator committee verifies the witness without holding the full shard state
- Decouples storage requirements from validation



NEAR's scalability achievement

Property	Value
Block time	1 s
Intra-shard finality	~1-2 s
Cross-shard finality	2+ blocks (~2 s minimum per hop)
Throughput scaling	Linear with shard count (theoretical)
Practical mainnet TPS	Demand-limited at current load (well below 4000 TPS capacity) and 9 shards

Tradeoffs:

- *Fundamental architectural constraint:* async receipts mean cross-shard composability carries inherent latency.
- Transactions are not atomic.
- Heavy reliance on snapshotting and archiving.

High-throughput substrate chains

Chain	Consensus	TPS (measured/claimed)	Block time	Notes
Aleph Zero	AlephBFT (DAG+BFT)	~89,600 (2021 prototype, 112 nodes)	~0.4 s finality	Golang prototype; ZK-shielded (LIMINAL) ~10–50 TPS
Polkadot + parachains	BABE+GRANDPA +Parachain	143,343 aggregate (Spammening 2024, ~23% of cores)	~6 s relay	On Kusama canary network; single parachain ~1,000–2,000 TPS
Acala	AURA+GRANDPA (parachain)	~200 TPS	~12 s	DeFi ops; relay-chain bound pre-async-backing
Moonbeam	AURA+GRANDPA (parachain)	~30–60 TPS max; ~0.7 TPS mainnet avg	~6 s	EVM compatibility traded for throughput
Astar	AURA+GRANDPA (parachain)	~1,000 TPS claimed WASM; mainnet ~100–500	~6 s	EVM + WASM; ZK Dapp support planned

- While several ZK blockchains have the technical capacity to exceed 100 TPS, **sustained organic demand** for ZK-proof-bearing transactions in production rarely exceeds a **network-wide average of 50 TPS** over long time horizons.
- Many of the publicized high TPS ZK chains involve L2 sequencers or batch aggregation.
- It is only since the rollout of next-generation proof systems and data availability upgrades in 2024 and 2025 that high-throughput ZK execution has become both technically and economically viable for public mainnets.

Lessons from Ouroboros Leios

Area	Finding
Full Leios sharding	Community found required features undesirable: failed transactions recorded in blocks; fees/collateral forfeit on conflict; DoS risks from sharding; shard-selection tokens or flavored UTxOs; cascading impacts to indexers, wallets, and backends
TCP multiplexing	Praos's multiplexing of mini-protocols over TCP created problems with timely delivery of Praos blocks in the Leios prototype
Simulator optimism	Two Leios simulators (low-fidelity and medium-fidelity for TCP) may be overly optimistic about throughput achievable in a more realistically engineered prototype
Testing infrastructure	Either actual large-scale deployments or sophisticated network-emulation infrastructure (e.g. Antithesis) is needed to study real node behavior at high throughput; simple network-manipulation tools are inadequate
Tail risk	Latency/bandwidth tail risks are significant for maintaining throughput and/or consensus
Design space	Both network and CPU resources are involved in the design space; such tradeoffs can be studied with constraint modeling

Midnight's binding constraint

Resource	Limit per block	Ceiling at 6 s slot
Block size	200 KB	~3.3 TPS (200 KB $\div$ ~10 KB/tx) ← binding
Compute budget	1 s (single-threaded)	~49 TPS (1,000 ms $\div$ 3.4 ms/proof)
Proof verification	~3.4 ms constant + ~3.4 μ s/public-input	—

- Block size is the binding constraint — roughly **15× tighter** than the compute ceiling.
- Switching to NEAR's networking does not address the bottleneck.
- ZK blockchains typically do not sustain more than ~50 TPS for ZK-proof-bearing transactions in production.

Applicability of throughput techniques

Technique	Impact on TPS	ZK-compatible?	Cost	Verdict
Block size increase (parameter)	~3–25×	Yes	Trivial	Immediate win; already identified; enabled by TCP/QUIC improvement
Shorter AURA slot time (parameter)	~2–3×	Yes	Low	Multiplies block-size gain; evaluate jointly
ZK proof aggregation (ProtoGalaxy)	~10× on compute ceiling	Yes (ZK-native)	High	High ROI; non-trivial; blocked pending security audit
Batch proof verification host function	~N× on compute ceiling	Yes (native Rust)	Medium	Architecturally clean for Midnight; not yet in Substrate
AlephBFT consensus	~3–5× (slot reduction)	Yes (consensus only)	Medium–High	Open-source; subsecond finality; no runtime changes
Parallel extrinsic execution	Potentially large	N/A	N/A — does not exist	Not available in Substrate
External ZK pallet (pallet-plonk, zkVerify)	Negative (slower)	Inferior	—	Do not adopt; native host function is faster

ProtoGalaxy folding risks

HYPOTHESIS: Replacing GRANDPA with AlephBFT and increasing block size to 5 MB achieves ~250 TPS with ~1 s finality, improving both throughput and UX without changing any Midnight pallet or ZK circuit.

Liveness risks:

- Aggregator is a new single point of failure; fallback to single-proof mode needed
- Batching latency at low traffic: waiting for $k=10$ proofs multiplies per-tx latency; timeout mechanism required
- Circuit-type fragmentation: ProtoGalaxy folds same-circuit proofs only; mixed blocks limit effective fold factor

Safety risks:

-  Formal security analysis of Midnight circuit compositions to be folded is required before production deployment — distinct from soundness of ProtoGalaxy itself
- Folded proof size may not be smaller than individual proofs; must be measured
- Soundness relies on KZG commitment hardness; the specific Midnight composition has not been independently audited.

TCP kernel tuning

Setting	Try now?	Try at 5 MB blocks?	Scope
<code>tcp_slow_start_after_idle = 0</code>	Yes — immediately	Yes	Docker <code>--sysctl</code>
<code>tcp_congestion_control = bbr</code>	Yes — immediately	Yes	Docker <code>--sysctl</code>
<code>default_qdisc = fq</code>	Yes — immediately	Yes	Docker <code>--sysctl</code> (Linux $\geq$ 4.18)
<code>tcp_mtu_probing = 1</code>	Yes (hygiene)	Yes	Docker <code>--sysctl</code>
<code>tcp_max_syn_backlog = 8096</code>	Yes (hygiene)	Yes	Docker <code>--sysctl</code>
<code>tcp_rmem & tcp_wmem_max = 8 MB</code>	Not required yet	Yes — prerequisite	Docker <code>--sysctl</code> (Linux $\geq$ 4.15)
<code>rmem_max & wmem_max = 8 MB</code>	Not required yet	Yes — if node calls <code>setsockopt</code>	Host root <code>sysctl.d</code>
<code>vm.swappiness = 1-10</code>	Yes (trie I/O stability)	Yes	Host root <code>sysctl.d</code>
<code>vm.dirty_ratio = 10-20</code>	Yes (trie I/O stability)	Yes	Host root <code>sysctl.d</code>
<code>net.core.netdev_max_backlog</code> increase	Yes (hygiene)	Yes	Host root <code>sysctl.d</code>
<code>fs.file-max</code> increase	Yes (operational)	Yes	Host root <code>sysctl.d</code>
<code>ulimit -n</code> (open file descriptors)	Yes (operational)	Yes	Docker <code>--ulimit nofile=65536:65536</code>

QUIC transport for Midnight Nodes

Property	TCP + Yamux (current)	QUIC (via litep2p)
Head-of-line blocking	All streams stall on any packet loss	Streams independent; lost packet blocks only its stream
Handshake latency	2.5 RTT (TCP + TLS 1.3)	1 RTT new; 0-RTT resumption for known peers
Slow start after idle	<code>tcp_slow_start_after_idle</code> resets CWND; ~700 ms penalty at 5 MB	Larger initial CWND + pacing; structurally absent
Stream multiplexing	Yamux over TCP; large block transfer delays GRANDPA votes	Native independent streams; GRANDPA unaffected by concurrent block download
NAT traversal	DCUtR hole punching	Connection migration via connection IDs; survives IP change

Throughput-technique risk summary

Technique	Primary liveness risk	Primary safety risk	Centralisation pressure
Block size increase	Diffusion failure at large sizes / short slots	DoS amplification; no native erasure coding	High (favours high-bandwidth validators)
Shorter slot time	Tightened diffusion window; higher fork rate	Higher steady-state fork exposure; bridge complexity	Moderate (requires low-latency nodes)
(5 MB, 2 s) combined	Marginal $\Delta_{\text{diff}}/\Delta_{\text{slt}}$ ratio; tail validators excluded	Compounded fork + diffusion risks	High — consider (5 MB, 3 s) as safer initial target
ProtoGalaxy folding	Aggregator as new SPF; batching latency at low load; circuit fragmentation	Folding-relation soundness; independent audit required	Low (aggregator can be decentralised)
Batch proof verification	Thread contention with networking under full load	Batch randomness must be unpredictable to submitters; weight model re-calibration	Low (purely internal to node)
AlephBFT consensus	DAG round failure if committee members offline	< 1/3 adversarial threshold; newer codebase	Moderate (rotating committee)

Findings / hypotheses

1. Empirical measurements, though costly, are essential even in early stages of a potential throughput upgrade.
2. Apparently, there are no general-purpose network simulators/emulators available for Substrate, though a tool like Shadow or Antithesis might be useful.
3. The following will cumulatively support 150+ TPS.*
 - a. Block size increase (increased data rate)
 - b. Shorter slot time (increased data rate)
 - c. TCP tuning (increased reliability)
 - d. QUIC instead of TCP (enables larger blocks)
 - e. AlephBFT instead of GRANDPA (enables shorter slot times)
 - f. Batch proof verification (quickens verification)
 - g. Protogalaxy folding (removes CPU bottleneck)

** Have any of these been evaluated already?*

NEXT STEPS

Major tentative findings / hypotheses

Hypothesis	Pro	Con
Most of NEAR's key management can be emulated by contract patterns / circuit libraries in Midnight.	Minimal or no impact on ledger	Requires excellent DX for adoption
W3C DIDs provide a superset of the named-account functionality.	Standardization and interoperability	Less straightforward for users
Midnight intents will support metatransactions.	Work already underway	Cryptographic complexity
NEAR's architecture and codebase do not provide sufficient enhancements to justify migrating Midnight to it.	Some NEAR capabilities portable and others emulatable	NEAR does have a superior user experience.
NEAR's architecture and design seem less demonstrably robust than Substrate.	NEAR is not formalized, more often refactored, and less audited	NEAR has demonstrated continuous operation without major incident.
Cryptographic incompatibilities prevent wholesale adoption of NEAR chain signatures.	Signing-path incompatibility is non-trivial	Possible resolution by adapting protocol or FROST
A combination of specific parameter and protocol changes achieves 150+ TPS.	Minimal impact on architecture and Substrate usage	Very labor- and resource-intensive
NEAR's notable UX and DX can be ported to Midnight.	Generally minimal impact on overall architecture.	Labor intensive

Feedback, Collaboration, and Next Increment

1. **Candid feedback** — What is most and least useful from the work so far? What is missing?
2. **Tighter collaboration** — Who else should be involved? What access, context, or artefacts would accelerate the next increment?
3. **Course corrections** — Are the three options still the right frame? Are any assessment areas over- or under-weighted?
4. **Next increment** — What should be prioritised for the Define phase (Apr 6 – Apr 30)?

Possible Next Steps

1. **Scalability experiments** — measure actual Midnight ZK transaction size and TPS on a testnet; benchmark block propagation at larger block sizes; validate the throughput model against real numbers.
2. **Broaden assessment** — chain signatures, TEE integration, Starstream/Nightstream details, Paima L2 path.
3. **Deep dive(s)** — select one or more topics for more detailed design and prototyping (e.g., ProtoGalaxy folding, chain signatures, AlephBFT).
4. **Define decision criteria** — agree in advance what evidence would be sufficient to recommend Option 1, 2, or 3; without this the Define phase may produce analysis without a path to a recommendation.
5. **Pivot?** — reconsider scope, framing, or which options remain live given findings to date.